Федеральное государственное бюджетное образовательное учреждение высшего образования
«Херсонский государственный педагогический университет»

Институт информационных технологий

Кафедра программной инженерии и информационных технологий

ПРАКТИЧЕСКАЯ РАБОТА
по дисциплине:
«Процессы и профессиональная практика программной инженерии»

на тему:
«Практическая работа 7. Структурный рефакторинг»

Выполнил:
Студент гр.: 12-25РПм
Трифонов Д.С.

Проверил
Преподаватель:
Алешов Е.В.

Оценка:_____
Дата: 10.11.2026

Херсон, 2026

ПРАКТИЧЕСКАЯ РАБОТА

1. Цель работы

Цель работы — на вымышленных данных учебной системы обмена исходящими сообщениями ООО «Аврора Логистик» продемонстрировать методы структурного рефакторинга (Extract Class, Subsume Method, Extract Interface) и паттерны рефакторинга корпоративных приложений Identity Map и Data Mapper, сохранив внешнее поведение программы и улучшив её внутреннюю структуру.

2. Исходные данные (вымышленные)

В качестве предметной области использован упрощённый аналог учебного примера EM из методических материалов. Данные хранятся в JSON-файлах каталога app/data/:

• сотрудники: Смирнова А.В. (логистика), Козлов М.И. (продажи), Волкова Е.С. (IT);

• исходящие сообщения №1001–1005 с адресатами ivanov@aurora-log.ru, petrova@aurora-log.ru, sidorov@nord-trade.ru и др.;

• деловые партнёры (контакты) Иванов П.А. и Сидоров Н.П.

Программная реализация размещена в каталоге app/: вариант «до» — before/system.py, вариант «после» — пакеты after/entity, after/control, after/presentation, after/mediator, after/foundation. Запуск демонстрации: python demo.py.

3. Ход выполнения работы

3.1. Анализ «дурно пахнущего» кода. В исходном варианте класс CActioner совмещал поиск и отправку сообщений (большой класс), методы retrieve_message и retrieve_messages дублировали фильтрацию, а класс PConsole инкапсулировал меню, список сообщений и отображение текста письма.

```
Структура проекта app/

app/
├── before/
│   └── system.py
├── after/
│   ├── entity/
│   │   ├── models.py
│   │   └── identity_map.py
│   ├── control/
│   │   ├── admin.py
│   │   ├── msg_seeker.py
│   │   └── msg_sender.py
│   ├── presentation/
│   │   ├── interfaces.py
│   │   ├── menu.py
│   │   └── viewers.py
│   ├── mediator/
│   │   ├── data_mapper.py
│   │   └── moderator.py
│   └── foundation/
│       └── io.py
├── data/
│   ├── employees.json
│   ├── messages.json
│   └── contacts.json
├── demo.py
└── make_screenshots.py
```

Рисунок 1 — Структура каталога app/ после выделения слоёв PCMEF

3.2. Проверка внешнего поведения до рефакторинга. Для учётной записи a.smirnova программа вывела неотправленные сообщения #1001 и #1002, показала текст письма #1001 и имитировала отправку по электронной почте. После отправки в списке осталось только сообщение #1002 — это эталон внешнего поведения.

```
Запуск: код до рефакторинга

1. КОД ДО РЕФАКТОРИНГА (запахи большого класса и дублирования)

ООО «Аврора Логистик» — демо ДО рефакторинга

Вход выполнен: Смирнова Анна Викторовна (Отдел логистики)

=== Меню (до рефакторинга) ===
Неотправленные сообщения:
  #1001 | ivanov@aurora-log.ru | График поставок на ноябрь
  #1002 | petrova@aurora-log.ru | Акт сверки за октябрь

=== Меню (до рефакторинга) ===
Текст сообщения #1001
Кому: ivanov@aurora-log.ru
Тема: График поставок на ноябрь
---
Добрый день! Просим подтвердить график поставок на ноябрь 2026.

=== Меню (до рефакторинга) ===
[EMAIL] Отправлено «График поставок на ноябрь» → ivanov@aurora-log.ru

=== Меню (до рефакторинга) ===
Неотправленные сообщения:
  #1002 | petrova@aurora-log.ru | Акт сверки за октябрь
```

Рисунок 2 — Запуск монолитного варианта (до рефакторинга)

Фрагмент CActioner / PConsole до рефакторинга

```
055|         self._messages: list[EOutMessage] = []
056|         self._load_messages()
057|
058|     def _load_messages(self) -> None:
059|         raw = json.loads((DATA_DIR / "messages.json").read_text(encoding="utf-8"))
060|         self._messages = [EOutMessage(**item) for item in raw]
061|
062|     def login(self, login: str) -> EEmployee | None:
063|         employees = json.loads((DATA_DIR / "employees.json").read_text(encoding="utf-8"))
064|         for item in employees:
065|             if item["login"] == login:
066|                 self._employee = EEmployee(**item)
067|                 return self._employee
068|         return None
069|
070|     def retrieve_messages(self, only_unsent: bool = True) -> list[EOutMessage]:
071|         """Извлечение нескольких исходящих сообщений."""
072|         if self._employee is None:
073|             return []
074|         result: list[EOutMessage] = []
075|         for msg in self._messages:
076|             if msg.employee_oid != self._employee.oid:
077|                 continue
078|             if only_unsent and msg.sent:
079|                 continue
080|             result.append(msg)
081|         return result
082|
083|     def retrieve_message(self, msg_id: int) -> EOutMessage | None:
084|         """Дублирует логику фильтрации из retrieve_messages."""
085|         if self._employee is None:
086|             return None
087|         for msg in self._messages:
088|             if msg.employee_oid != self._employee.oid:
089|                 continue
090|             if msg.msg_id == msg_id:
091|                 return msg
092|         return None
093|
094|     def send_message(self, msg_id: int) -> str:
095|         """Отправка исходящего сообщения (смешана с поиском в одном классе)."""
096|         msg = self.retrieve_message(msg_id)
097|         if msg is None:
098|             return f"Сообщение {msg_id} не найдено"
099|         if msg.sent:
100|             return f"Сообщение {msg_id} уже отправлено ранее"
101|         msg.sent = True
102|         return f"[EMAIL] Отправлено «{msg.subject}» → {msg.to}"
103|
104|
105| class PConsole:
106|     """Большой класс уровня presentation: меню + список + тело письма."""
107|
108|     def __init__(self, actioner: CActioner) -> None:
109|         self._actioner = actioner
110|
```

Рисунок 3 — Фрагмент классов CActioner и PConsole до рефакторинга

3.3. Extract Class. Класс CActioner переименован в CAdmin; обязанности поиска и отправки вынесены в CMsgSeeker и CMsgSender. Связи поддерживаются методами get_msg_seeker() и get_msg_sender(), ассоциация служащего задаётся через set_employee() после login().

3.4. Subsume Method. Метод retrieve_message() устранён: его поведение включено в retrieve_messages(..., msg_id=...). Различие «несколько сообщений / одно сообщение» сохранено на уровне presentation.

3.5. Extract Interface. На уровне presentation из PConsole выделены PMenu, POutMsgListViewer и POutMsgBodyBuilder; общий метод display() вынесен в интерфейс IDisplay. Это упрощает последующий переход к GUI.

Extract Class + Subsume Method

```python
"""CAdmin — результат Extract Class из CActioner."""

from __future__ import annotations

from after.entity import EEmployee
from after.mediator import MModerator

from .msg_seeker import CMsgSeeker
from .msg_sender import CMsgSender


class CAdmin:
    def __init__(self, moderator: MModerator | None = None) -> None:
        self._moderator = moderator or MModerator()
        self._seeker = CMsgSeeker(self._moderator)
        self._sender = CMsgSender(self._moderator)
        self._employee: EEmployee | None = None

    def login(self, login: str) -> EEmployee | None:
        employee = self._moderator.get_employee_by_login(login)
        if employee is None:
            return None
        self._employee = employee
        self._seeker.set_employee(employee)
        return employee

    def get_msg_seeker(self) -> CMsgSeeker:
        return self._seeker

    def get_msg_sender(self) -> CMsgSender:
        return self._sender
```

```python
"""Extract Class: поиск исходящих сообщений."""

from __future__ import annotations

from after.entity import EEmployee, EOutMessage
from after.mediator import MModerator


class CMsgSeeker:
    def __init__(self, moderator: MModerator) -> None:
        self._moderator = moderator
        self._employee: EEmployee | None = None

    def set_employee(self, employee: EEmployee) -> None:
        self._employee = employee

    def retrieve_messages(
        self,
        only_unsent: bool = True,
        msg_id: int | None = None,
    ) -> list[EOutMessage]:
        """
        Subsume Method: прежние retrieve_message и retrieve_messages
        объединены в один метод с опциональным msg_id.
        """
        if self._employee is None:
            return []
        return self._moderator.get_messages(
            employee_oid=self._employee.oid,
            only_unsent=only_unsent if msg_id is None else False,
            msg_id=msg_id,
        )
```

Рисунок 4 — Результат Extract Class и Subsume Method

3.6. Проверка внешнего поведения после рефакторинга. Сценарий входа, просмотра списка, текста письма и отправки сообщения #1001 дал тот же результат, что и до рефакторинга: поведение сохранено, структура кода улучшена.

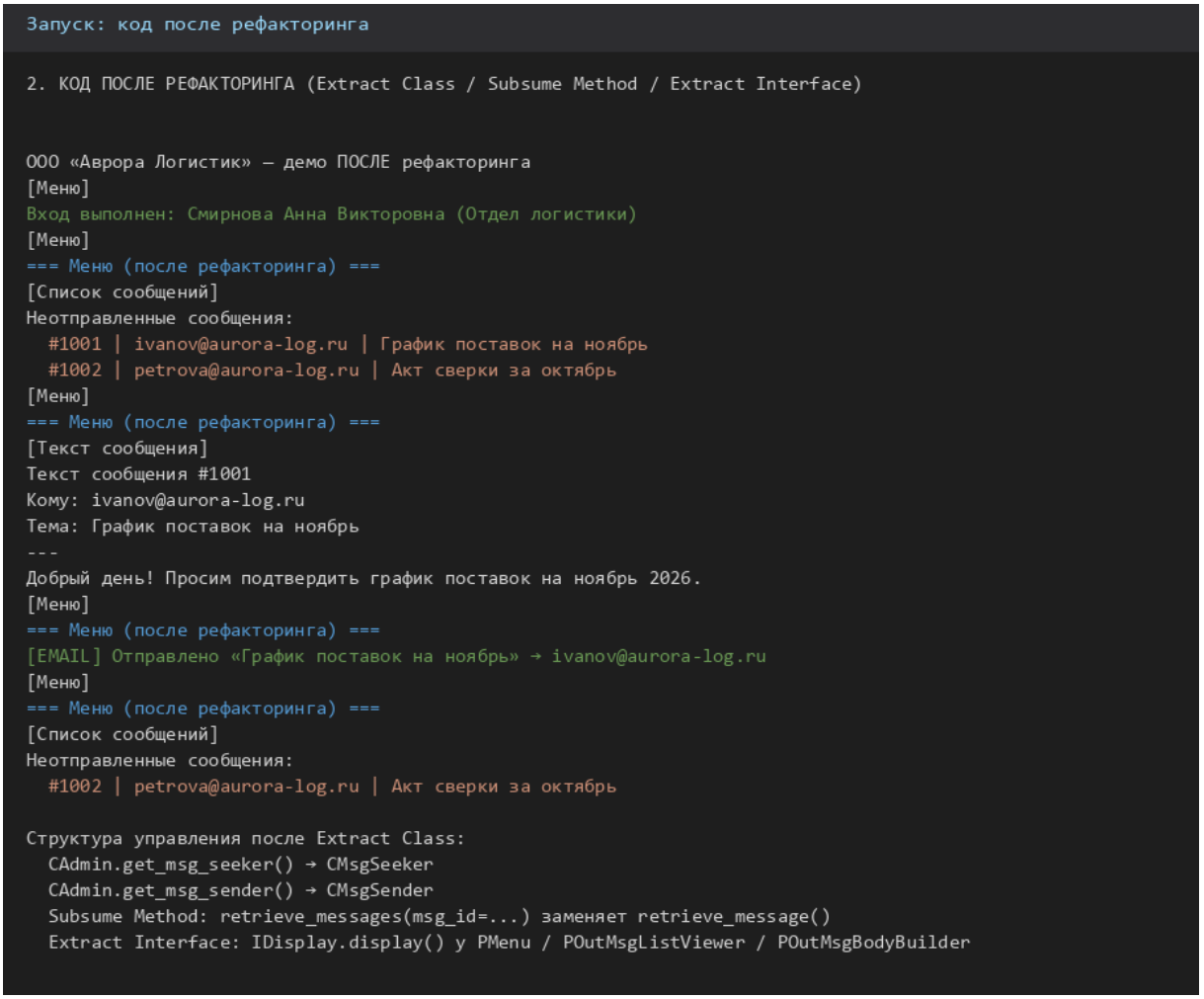


Рисунок 5 — Запуск рефакторингового варианта (после изменений)

3.7. Паттерн Identity Map. Класс EIdentityMap хранит загруженные объекты entity по OID и отдельно отображает msgID → OID для EOutMessage. Повторный retrieve служащего a.smirnova возвращает тот же экземпляр из кэша (id совпадает).

3.8. Паттерн Data Mapper. Класс MDataMapper реализует интерфейс IMDataMapper (get_by_oid, retrieve, insert, update, delete), отделяет entity-кэш от доступа к «БД» через FReader/FWriter и выполняет загрузку (check-out) и выгрузку (check-in) исходящих сообщений. Фасадом пакета mediator служит MModerator.

Extract Interface и Identity Map

```
"""Extract Interface: общий метод display() для уровня presentation."""

from __future__ import annotations

from abc import ABC, abstractmethod


class IDisplay(ABC):
    @abstractmethod
    def display(self, text: str) -> str:
        """Показать строку на экране."""
        raise NotImplementedError
```

```
"""Паттерн Identity Map (Коллекция идентичности объектов)."""

from __future__ import annotations

from typing import Any

from .models import EContact, EEmployee, EOutMessage, IEObjectID


class EIdentityMap:
    """Явно заданная коллекция: отдельный API для каждого класса entity."""

    def __init__(self) -> None:
        self._oid_to_obj: dict[int, Any] = {}
        self._msg_pk_to_oid: dict[int, int] = {}

    def find_employee(self, employee_oid: int) -> EEmployee | None:
        obj = self._oid_to_obj.get(employee_oid)
        return obj if isinstance(obj, EEmployee) else None

    def register_employee(self, employee: EEmployee) -> None:
        self._oid_to_obj[employee.get_oid()] = employee

    def unregister_employee(self, employee: IEObjectID) -> None:
        self._oid_to_obj.pop(employee.get_oid(), None)

    def find_contact(self, contact_oid: int) -> EContact | None:
        obj = self._oid_to_obj.get(contact_oid)
        return obj if isinstance(obj, EContact) else None

    def register_contact(self, contact: EContact) -> None:
        self._oid_to_obj[contact.get_oid()] = contact

    def unregister_contact(self, contact: IEObjectID) -> None:
        self._oid_to_obj.pop(contact.get_oid(), None)

    def find_message(self, message_oid: int) -> EOutMessage | None:
        obj = self._oid_to_obj.get(message_oid)
        return obj if isinstance(obj, EOutMessage) else None

    def find_message_by_pk(self, msg_id: int) -> EOutMessage | None:
        oid = self._msg_pk_to_oid.get(msg_id)
        if oid is None:
            return None
        return self.find_message(oid)
```

Рисунок 6 — Extract Interface (IDisplay) и Identity Map

Интерфейс IMDataMapper / MDataMapper

```python
"""Паттерн Data Mapper (Преобразователь данных)."""

from __future__ import annotations

from abc import ABC, abstractmethod
from typing import Any

from after.entity import EContact, EEmployee, EIdentityMap, EOutMessage
from after.foundation import FReader, FWriter


class IMDataMapper(ABC):
    @abstractmethod
    def get_by_oid(self, oid: int) -> Any:
        raise NotImplementedError

    @abstractmethod
    def retrieve(self, entity_type: str, **criteria: Any) -> Any:
        raise NotImplementedError

    @abstractmethod
    def insert(self, entity: Any) -> None:
        raise NotImplementedError

    @abstractmethod
    def update(self, entity: Any) -> None:
        raise NotImplementedError

    @abstractmethod
    def delete(self, entity: Any) -> None:
        raise NotImplementedError


class MDataMapper(IMDataMapper):
    """Единый преобразователь данных для entity-объектов итерации 1."""

    def __init__(self, identity_map: EIdentityMap | None = None) -> None:
        self._map = identity_map or EIdentityMap()
        self._reader = FReader()
        self._writer = FWriter()
        self._cache_dirty = False
        self._own_oids: set[int] = set()

    @property
    def identity_map(self) -> EIdentityMap:
        return self._map

    def flag_cache(self) -> None:
        self._cache_dirty = True

    def get_by_oid(self, oid: int) -> Any:
        cached = self._map.find_employee(oid) or self._map.find_contact(oid) or self._map.find_message(oid)
        if cached is not None and not self._cache_dirty:
            return cached
        return self.retrieve("any", oid=oid)
```

Рисунок 7 — Фрагмент интерфейса IMDataMapper / MDataMapper

Identity Map и Data Mapper

```
3. ПАТТЕРНЫ Identity Map + Data Mapper


Загрузка служащего a.smirnova (материализация / check-out):
  Получен объект: EEmployee(oid=101, login='a.smirnova', full_name='Смирнова Анна Викторовна', department='Отдел логистики', email='a.smirnova@aurora-log.ru')
  Размер кэша Identity Map: 1
  Содержимое кэша:
    OID=101 → EEmployee

Повторный retrieve того же login берёт объект из кэша (без лишней загрузки):
  Тот же OID=101, id(obj) совпадает: True

Контакт из «БД»: Сидоров Николай Павлович <sidorov@nord-trade.ru>, ООО «Норд-Трейд»
  Размер кэша после регистрации контакта: 2

Неотправленные сообщения Смирновой: 2 шт.
  Выгрузка (check-in): помечаем сообщение #1001 как отправленное
  [EMAIL] Отправлено «График поставок на ноябрь» → ivanov@aurora-log.ru
  Кэш после update/unregister: 3 объект(ов)
  Снимок кэша:
    OID=101 → EEmployee
    OID=202 → EContact
    OID=502 → EOutMessage

Методы IMDataMapper: get_by_oid, retrieve, insert, update, delete — реализованы в MDataMapper.

[лог сохранён] C:\Users\mitriy\Desktop\task-ref\app\screenshots\demo_output.txt
```

Рисунок 8 — Демонстрация Identity Map и Data Mapper на вымышленных данных

4. Результаты работы

В результате выполнения практической работы получено следующее:

1) построена демонстрационная программа на вымышленных данных ООО «Аврора Логистик» с вариантами «до» и «после» структурного рефакторинга;

2) применены методы Extract Class, Subsume Method и Extract Interface без изменения внешнего поведения пользовательского сценария;

3) реализованы паттерны Identity Map и Data Mapper (включая материализацию/пассивацию объектов и методы интерфейса IMDataMapper);

4) подтверждено, что повторная загрузка сущности по OID/login использует кэш, а отправка сообщения корректно обновляет хранилище и состояние кэша.

5. Ответы на контрольные вопросы

1. Что понимается под термином «рефакторинг», и какова его главная цель по отношению к внешнему поведению и внутренней структуре ПО?

Рефакторинг — изменение системы ПО таким образом, что внешнее поведение кода не меняется, но улучшается его внутренняя структура. Главная цель — устранить «дурные запахи» и повысить понятность, сопровождаемость и масштабируемость без изменения наблюдаемого результата для пользователя.

2. Какие изменения в связях между классами и классах-клиентах необходимо внести после применения рефакторинга Extract Class?

После извлечения полей и методов в новый класс между старым и новым классом устанавливается ассоциация (часто односторонняя). Клиенты, зависевшие от перенесённых методов, должны обращаться к ним через новый класс или через методы-посредники старого класса (в работе — get_msg_seeker()/get_msg_sender()).

3. В чем состоит назначение метода рефакторинга «Метод подключения» (Subsume Method) и чем он отличается от «Метода извлечения» (Extract Method)?

Subsume Method устраняет метод, включая его функциональность в уже существующий метод (в работе retrieve_message включён в retrieve_messages). Extract Method, напротив, выделяет повторяющийся фрагмент в новый отдельный метод.

4. Как совместное применение Extract Class и Extract Interface на уровне presentation помогает подготовить систему к переходу от консоли к GUI?

Extract Class разделяет обязанности отображения (меню, список, тело письма), а Extract Interface фиксирует общий контракт display(). При смене UI достаточно заменить реализации интерфейса, не переписывая логику control/mediator.

5. Что представляют собой паттерны рефакторинга и какие проблемы они решают?

Это структурные паттерны корпоративных приложений (по М. Фаулеру): работа с БД, кэширование объектов в памяти, транзакции и параллелизм, Web и распределённые объекты. Они повышают масштабируемость при переходе от учебного прототипа к промышленному решению.

6. Каковы роли процессов загрузки (материализации / check-out) и выгрузки (дематериализации / check-in) в контексте Data Mapper?

Загрузка читает запись БД и создаёт объект в памяти с регистрацией в Identity Map. Выгрузка сохраняет новый/изменённый объект в БД или удаляет запись и снимает объект с регистрации в кэше (в демо — update исходящего сообщения после отправки).

7. В каких случаях единственный универсальный Data Mapper становится неэффективным и требует разделения?

Когда число классов entity велико, а способы запросов и наборы полей сильно различаются, единый преобразователь «раздувается». Тогда применяют по одному Data Mapper на класс entity (или метаданные/динамическое формирование).

8. Перечислите базовые методы IMDataMapper и опишите их назначение.

get_by_oid — получить объект по OID (из кэша или БД); retrieve — извлечь по критериям и при необходимости материализовать; insert — сохранить новый объект; update — записать изменения; delete — удалить объект из БД и кэша.

9. В чем цель паттерна «Преобразователь метаданных» и какие преимущества он даёт при изменении схемы БД?

Паттерн динамически строит объектно-реляционное соответствие по метаданным (таблицы↔классы, столбцы↔поля). При изменении схемы достаточно обновить метаданные, а не переписывать множество жёстко заданных преобразователей.

10. Назовите три способа хранения метаданных, их достоинства и недостатки.

1) В исходном коде — просто и прозрачно, но правки требуют перекомпиляции. 2) Во внешнем XML/файле — гибкая настройка без пересборки, нужна загрузка и валидация при старте. 3) В БД — единый источник для нескольких приложений и автопроверка схемы, но выше связность с инфраструктурой хранения.

6. Вывод

Структурный рефакторинг позволил устранить запахи большого класса и дублирования кода в демонстрационном приложении ООО «Аврора Логистик», сохранив пользовательский сценарий. Паттерны Identity Map и Data Mapper обеспечили корректное кэширование сущностей и отделение объектной модели от доступа к данным. Полученная структура соответствует принципам PCMEF и готовит уровень presentation к дальнейшему развитию.